

Primer query: extraer los 12 meses de ambos parties

Reemplaza solamente los dos IDs:

```
party_month_audit <- sdf_sql(  
  sc,  
  "  
  SELECT  
    party_id,  
    observation_date,  
    label_3m,  
    feature_month,  
    month_offset,  
    feature_window,  
  
    active_accounts,  
    avg_balance_active,  
    avg_past_due_active,  
    delinquent_account_share,  
    avg_utilization,  
  
    deterioration_count,  
    adverse_entry_count,  
    max_dpd_jump  
  
  FROM bu_frd.model_party_behavior_monthly_2022_2025  
  
  WHERE party_id IN (  
    <PARTY_ID_LABEL_1>,  
    <PARTY_ID_LABEL_0>  
  )  
  
  ORDER BY  
    party_id,  
    month_offset DESC  
  "  
) %>%  
  collect()
```

party_month_audit

Aquí deberíamos obtener:

$$12 \times 2 = 24$$

filas.

Segundo query: utilización ponderada

Como weighted utilization vive en su tabla mensual:

```
party_utilization_audit <- sdf_sql(  
  sc,  
  "  
  SELECT  
    party_id,  
    observation_date,  
    feature_month,  
    month_offset,  
  
    avg_utilization,  
    weighted_utilization  
  
  FROM bu_frd.model_party_utilization_monthly_2022_2025  
  
  WHERE party_id IN (  
    <PARTY_ID_LABEL_1>,  
    <PARTY_ID_LABEL_0>  
  )  
  
  ORDER BY  
    party_id,  
    month_offset DESC  
  "  
) %>%  
  collect()  
  
party_utilization_audit
```

Tercer query: valores finales del database

Este es fundamental para comparar el cálculo manual con la base final:

```
party_final_features <- sdf_sql(  
  sc,  
  "  
  SELECT  
    party_id,
```

observation_date,
label_3m,

deterioration_acceleration,
adverse_acceleration,
max_dpd_jump_recent_3m,

active_accounts_change_vs_baseline,
active_accounts_changed_flag,

balance_change_model,
log1p_abs_balance_change_model,
balance_change_available_flag,
balance_recent_months_available,
balance_prior_months_available,

past_due_change_model,
past_due_change_available_flag,
past_due_recent_months_available,
past_due_prior_months_available,

delinquency_share_change_model,
delinquency_change_available_flag,
delinquency_recent_months_available,
delinquency_prior_months_available,

avg_utilization_change_model,
weighted_utilization_change_model,
utilization_change_available_flag,
utilization_recent_months_available,
utilization_prior_months_available,

active_accounts_z_model,
active_accounts_z_available_flag,

avg_utilization_z_model,
avg_utilization_z_available_flag,

weighted_utilization_z_model,
weighted_utilization_z_available_flag

FROM bu_frd.model_features_ab_final_2022_2025

WHERE party_id IN (
 <PARTY_ID_LABEL_1>,
 <PARTY_ID_LABEL_0>
)
"

```
) %>%  
collect()
```

```
party_final_features
```

Cuarto query: construir la tabla de cálculo de los Current features

Este query ya resume Prior9 y Recent3.

```
party_calculation_audit <- sdf_sql(  
  sc,  
  "  
  WITH selected AS (  
  
    SELECT  
      *  
  
    FROM bu_frd.model_party_behavior_monthly_2022_2025  
  
    WHERE party_id IN (  
      <PARTY_ID_LABEL_1>,  
      <PARTY_ID_LABEL_0>  
    )  
  ),  
  
  agg AS (  
  
    SELECT  
      party_id,  
      observation_date,  
      label_3m,  
  
      /* ACTIVE ACCOUNTS */  
  
      AVG(  
        CASE  
          WHEN feature_window = 'PRIOR_9M'  
          THEN active_accounts  
        END  
      ) AS active_prior_avg,  
  
      STDDEV_SAMP(  
        CASE
```

```

        WHEN feature_window = 'PRIOR_9M'
        THEN active_accounts
    END
) AS active_prior_std,

AVG(
    CASE
        WHEN feature_window = 'RECENT_3M'
        THEN active_accounts
    END
) AS active_recent_avg,

/* BALANCE */

AVG(
    CASE
        WHEN feature_window = 'PRIOR_9M'
        THEN avg_balance_active
    END
) AS balance_prior_avg,

AVG(
    CASE
        WHEN feature_window = 'RECENT_3M'
        THEN avg_balance_active
    END
) AS balance_recent_avg,

/* PAST DUE */

AVG(
    CASE
        WHEN feature_window = 'PRIOR_9M'
        THEN avg_past_due_active
    END
) AS past_due_prior_avg,

AVG(
    CASE
        WHEN feature_window = 'RECENT_3M'
        THEN avg_past_due_active
    END
) AS past_due_recent_avg,

/* DELINQUENCY */

```

```
AVG(  
  CASE  
    WHEN feature_window = 'PRIOR_9M'  
    THEN delinquent_account_share  
  END  
) AS delinquency_prior_avg,
```

```
AVG(  
  CASE  
    WHEN feature_window = 'RECENT_3M'  
    THEN delinquent_account_share  
  END  
) AS delinquency_recent_avg,
```

```
/* UTILIZATION */
```

```
AVG(  
  CASE  
    WHEN feature_window = 'PRIOR_9M'  
    THEN avg_utilization  
  END  
) AS avg_util_prior_avg,
```

```
STDDEV_SAMP(  
  CASE  
    WHEN feature_window = 'PRIOR_9M'  
    THEN avg_utilization  
  END  
) AS avg_util_prior_std,
```

```
AVG(  
  CASE  
    WHEN feature_window = 'RECENT_3M'  
    THEN avg_utilization  
  END  
) AS avg_util_recent_avg,
```

```
/* COUNTS */
```

```
SUM(  
  CASE  
    WHEN feature_window = 'PRIOR_9M'  
    THEN deterioration_count  
    ELSE 0  
  END
```

```

) AS deterioration_prior_count,

SUM(
  CASE
    WHEN feature_window = 'RECENT_3M'
    THEN deterioration_count
    ELSE 0
  END
) AS deterioration_recent_count,

SUM(
  CASE
    WHEN feature_window = 'PRIOR_9M'
    THEN adverse_entry_count
    ELSE 0
  END
) AS adverse_prior_count,

SUM(
  CASE
    WHEN feature_window = 'RECENT_3M'
    THEN adverse_entry_count
    ELSE 0
  END
) AS adverse_recent_count,

MAX(
  CASE
    WHEN feature_window = 'RECENT_3M'
    THEN max_dpd_jump
  END
) AS max_dpd_jump_recent

FROM selected

GROUP BY
  party_id,
  observation_date,
  label_3m
)

SELECT
  *,

  active_recent_avg
  - active_prior_avg
  AS active_change_recalc,

```

```

balance_recent_avg
- balance_prior_avg
AS balance_change_recalc,

LN(
  1 +
  ABS(
    balance_recent_avg
    - balance_prior_avg
  )
) AS log_abs_balance_change_recalc,

past_due_recent_avg
- past_due_prior_avg
AS past_due_change_recalc,

delinquency_recent_avg
- delinquency_prior_avg
AS delinquency_change_recalc,

avg_util_recent_avg
- avg_util_prior_avg
AS avg_util_change_recalc,

deterioration_recent_count / 3.0
-
deterioration_prior_count / 9.0
AS deterioration_acceleration_recalc,

adverse_recent_count / 3.0
-
adverse_prior_count / 9.0
AS adverse_acceleration_recalc,

CASE
  WHEN active_prior_std > 0
  THEN
    (
      active_recent_avg
      - active_prior_avg
    )
    /
    (
      active_prior_std / SQRT(3.0)
    )
  END AS active_z_raw_recalc,

CASE

```

```

    WHEN avg_util_prior_std > 0
    THEN
      (
        avg_util_recent_avg
        - avg_util_prior_avg
      )
      /
      (
        avg_util_prior_std / SQRT(3.0)
      )
    END AS avg_util_z_raw_recalc

FROM agg
"
) %>%
collect()

```

party_calculation_audit

Una precaución: los Z finales pueden estar **winsorized**, por lo que:

active_z_raw_recalc

puede no ser idéntico al `active_accounts_z_model` si el raw Z supera los caps. Eso es correcto y podemos mostrarlo.

Cómo lo presentaría en el Word

Yo pondría esta sección inmediatamente después de **Feature Formulas**, con el título:

Worked Calculation Examples

Y una introducción:

Two analytical observations were selected to demonstrate the feature-engineering calculations: one observation with `label_3m = 1` and one with `label_3m = 0`. Party identifiers are masked in the documentation. The objective is not to compare two individuals statistically, but to demonstrate how the documented formulas reproduce the values stored in the final database.

Esto es importante: **los dos parties son ejemplos de cálculo**, no evidencia de que todos los positivos se comportan como el positivo seleccionado.

Tabla para el party con Label 1

Feature	Prior 9M	Prior SD	Recent 3M	Calculation	Final Database Value	Match
Active Accounts	...	...	...	Recent – Prior	...	✓
Balance	...	—	...	Recent – Prior	...	✓
Log Abs Balance	—	—	—	$\ln(1 + \text{abs}(\text{change}))$	...	✓
Past Due	...	—	...	Recent – Prior	...	✓
Delinquency Share	...	—	...	Recent – Prior	...	✓
Avg Utilization	...	...	...	Recent – Prior	...	✓
Deterioration	prior count / 9	—	recent count / 3	Recent rate – Prior rate	...	✓
Adverse	prior count / 9	—	recent count / 3	Recent rate – Prior rate	...	✓
Max DPD Jump	—	—	max Recent3	max	...	✓

Y exactamente la misma estructura para **label_3m** =